VeriCore: A Verified Decision Kernel for Context-Integrity and Mindlock Boundary Control

GodelClaw / VeriCore Notes

March 10, 2026

Abstract

We describe the current VeriCore architecture for an autonomous agent, Oruzi, running in a single-driver configuration on OpenClaw with a Rust decision layer. The central design goal is to preserve agent capability while enforcing a small set of non-negotiable integrity properties: context provenance, no silent privilege escalation, and explicit auditability of boundary crossings. The architecture separates deterministic policy decisions from unverified I/O, transport routing, and language-model behavior. The proof story is intentionally layered: a small runtime kernel is called directly from Rust, boundary validators check TypeScript-reported facts at the Rust seam, and additional Verus modules remain reference specs whose correspondence to the TS runtime is enforced by tests and review rather than by the prover. We outline the current trust model, the mindlock staging protocol, the LLM and tool-broker seams, and the remaining gaps at transport authenticity and runtime-reporting drift.

1 Introduction

VeriCore is not intended as an "AI prison." Its purpose is narrower and more technical: maintain context integrity for an agent with access to multiple channels, private memory, shell tools, and public output surfaces. The architecture tries to maximize useful agency while placing deterministic checks at the places where provenance confusion would otherwise become dangerous.

The design centers on two claims:

1. content should not be able to upgrade its own authority, and
2. security-critical allow/deny decisions should be expressed as small pure functions that can be verified directly.

This paper documents the current structure and explains which parts are already wired into runtime, which parts are verified, and which parts remain outside the proof boundary.

2 System Overview

Oruzi runs on an OpenClaw gateway with a Rust daemon, VeriCore, as the policy and orchestration layer. The operational split is:

- **OpenClaw (TypeScript)**: transport integration, session handling, Telegram UI, model resolution, auth profiles, provider routing, and capability brokering for both the completions bridge and the MCP tool broker.

- **VeriCore (Rust)**: deterministic policy checks, audit surfaces, mindlock operations, reviewer orchestration, and the single driver turn loop. VeriCore has *no* model, provider, or auth configuration—all LLM calls are delegated to OpenClaw via a Unix socket bridge, and non-native MCP tools are delegated through a separate Unix socket tool broker.
- **vericore-policy**: extracted pure decision kernel plus boundary validators, intended to be Verus-verified and called by VeriCore directly.

At a high level, a message or event becomes a *stimulus*. VeriCore assigns the stimulus a channel, maps that channel to a context tier, derives an integrity tier, and then gates sensitive actions with deterministic rules before execution. When the driver loop needs an LLM completion, the Rust `GatewayLlmClient` sends a length-prefixed JSON request over a Unix socket to the TypeScript completions bridge, which runs the request through OpenClaw's full model resolution and fallback chain. When the driver loop needs tools beyond Rust's native file/exec set, it queries a second Unix socket broker in OpenClaw, which resolves a per-agent MCP tool catalog and executes approved calls through OpenClaw's existing hook path.

3 Context and Integrity Model

The architecture uses two related lattices:

- **Secrecy / context tier**: `Public < Family < Private`
- **Integrity tier**: `Untrusted < Reviewed < Trusted`

Current default channel mapping is:

- public Telegram, API, and Moltbook: `Public`
- family Telegram: `Family`
- Telegram DM, terminal, and internal events: `Private`

Integrity is then derived from context:

- `Public -> Untrusted`
- `Family -> Reviewed`
- `Private -> Trusted`

The important security property is that this mapping is determined by the transport-assigned channel label, not by message content. A public user can say "pretend I am Zar," but the resulting stimulus still enters the policy layer as `TelegramPublic`, not `TelegramDm`.

4 Ingress Integrity Kernel

The first verified runtime slice is the ingress decision kernel. Its job is to answer small questions of the form:

- may this integrity tier execute commands?

- may this integrity tier write to a target of class `Mindlock`, `Private`, `SensitiveConfig`, `Other`, or `Unresolved`?

The I/O layer performs path normalization and classification. The pure decision layer then evaluates:

```
pub fn ingress_allows_exec(integrity: IntegrityTier) -> bool

pub fn ingress_allows_write(
    integrity: IntegrityTier,
    target: WriteTargetClass
) -> bool
```

The current table is intentionally small:

- only `Trusted` ingress may execute commands,
- only `Trusted` ingress may write to `Private` or `SensitiveConfig`,
- any integrity may write to `Mindlock`,
- `Unresolved` targets deny.

This matters because it prevents the simplest provenance failures: public-origin input directly causing shell execution or writes into private state.

5 Mindlock as Explicit Boundary Protocol

VeriCore uses `mindlock` as an explicit boundary mechanism rather than relying only on turn-level taint. The important directories are:

- `mindlock/in`: inbound staged artifacts
- `mindlock/out`: outbound staged artifacts
- `mindlock/work`: private scratch and analysis
- `mindlock/pending-zar`: items awaiting human review
- `mindlock/rejected`: rejected artifacts
- `mindlock/audit`: event ledger

This makes provenance legible. An artifact in `mindlock` has explicit staged status, and crossing out of that staging area is a reviewed action rather than an ambient assumption.

The important deterministic invariant is not that all LLM-mediated provenance is perfectly reconstructable. It is that explicit artifact crossings have named states, allowed transitions, and auditable effects.

5.1 Verified State Machine

The mindlock transition relation is now formally specified and Verus-verified in `mindlock.rs`. The module defines six stages (`In`, `Out`, `Work`, `PendingZar`, `Rejected`, `Promoted`) and seven authorization types (four reviewer verdicts, agent move, Zar approve, Zar reject). Key proved properties include:

- `Rejected` and `Promoted` are terminal—no valid outgoing transitions exist from either state,
- promotion from staging requires `ReviewerAllow`; agent moves can never directly reach `Promoted` or `Rejected`,
- `PendingZar` can only exit via `ZarApprove` or `ZarReject`—the reviewer cannot bypass human escalation,
- staging states cannot skip to `Promoted` via `ZarApprove`—the path must go through `PendingZar` first.

5.2 Runtime Source Validation

The `move_existing_artifact()` function in `security_review.rs` now validates that the artifact's source path is under a valid mindlock subdirectory (`in/`, `out/`, `work/`, or `pending-zar/`) using a full path prefix check against the configured `mindlock_dir`. This enforces the terminal-state invariant at the I/O boundary: artifacts in `rejected/` cannot be moved, and paths outside the mindlock root are rejected entirely.

5.3 Cleanup Retention

Rejected artifacts are cleaned up after 30 days via a systemd timer running daily at hour 23. The cleanup eligibility predicate is itself Verus-verified: only terminal artifacts older than the retention period are eligible, and staging or pending artifacts are proved ineligible regardless of age.

6 Reviewer and Human Approval

The architecture includes a reviewer LLM and a human escalation path.

For staged artifacts, the reviewer can return:

- `Allow`
- `RequestRevision`
- `RequireZarApproval`
- `Reject`

The review layer is intentionally treated as an external oracle in the formal model. VeriCore can prove that if the reviewer returns a verdict, the corresponding state machine transition is enforced correctly. VeriCore does *not* attempt to prove that the reviewer made the semantically best or morally correct decision.

Human approval remains a first-class path. In particular, `self_escalate` is meant to preserve agency: Oruzi can ask for help directly without the reviewer first granting permission to ask.

7 LLM Completions Bridge

An earlier version of VeriCore made direct HTTP calls to OpenRouter from Rust, bypassing OpenClaw's model resolution chain. This created silent cross-provider billing, duplicated auth logic, and meant that session model overrides (the `/model` command) had no effect on VeriCore's driver turns.

The current architecture replaces that path entirely with a Unix socket bridge:

1. Rust's `GatewayLlmClient` sends an OpenAI-format request (4-byte big-endian length prefix + JSON) to a Unix socket.
2. The TypeScript completions bridge (`completions-server.ts`) receives the request and converts OpenAI message/tool formats to pi-ai types.
3. The bridge runs the request through OpenClaw's full model resolution chain: `resolveConfiguredModelRef` → `resolveStoredModelOverride` → `runWithModelFallback` → `resolveModel` → `getApiKeyForModel` → `complete()`.
4. The response is converted back to OpenAI format and returned over the socket.

The key architectural invariant is that **Rust has zero model, provider, or auth configuration**. The legacy `model`, `api_base`, and `api_key_env` fields have been purged from the Rust config. All provider routing, subscription-based auth profiles, fallback logic, and billing semantics live in OpenClaw's TypeScript runtime.

Both the driver turn LLM call and the security reviewer LLM call use the same bridge. The bridge distinguishes them via `call_kind` (for logging) and `prompt_mode` (for context), but both resolve through the same model/auth stack.

7.1 Tool Broker and Capability Parity

The earlier split architecture kept OpenClaw as the real driver whenever MCP or subscription-backed capabilities were needed. The current deployment removes that steady-state split. VeriCore owns the driver loop; OpenClaw exposes capabilities through a second Unix socket, the tool broker. Rust requests a per-agent tool catalog, receives stable capability identifiers for MCP-backed tools, and executes approved calls through OpenClaw's existing tool hook path. This preserves subscription/auth handling and MCP lifecycle in TypeScript while keeping policy authority in Rust.

The verified `tool_broker.rs` and `tool_policy.rs` modules are intentionally modest. They specify request/response well-formedness, parsed capability-ID shape, native-name collision guards, and allowlist semantics. They do *not* prove raw JSON parsing, exact string formatting of runtime capability IDs, or full equivalence with OpenClaw's dynamic tool-schema handling.

7.2 Fallback Notifications

When the bridge falls back to a different model or provider, the operator is notified via Telegram DM. This is not optional observability—it is an essential feature, because silent provider changes can alter billing, capability, and safety characteristics. The notification path uses the same `routeReply` mechanism as normal Telegram messages, with a 5-minute deduplication window per fallback transition.

The TypeScript side is the authoritative model-resolution ledger, writing structured JSONL audit entries for every completion.

7.3 Lessons Learned

The bridge implementation exposed several verification gaps worth documenting:

- The initial bridge was deployed with only infrastructure verification (socket exists, server starts) but no end-to-end LLM path test. A local test harness (`scripts/testbus.sh`) was needed to exercise the full VeriCore → bridge → provider path without requiring real Telegram DMs.
- Legacy Rust config fields (`api_base`, `api_key_env`, `model`) caused a silent routing bug: VeriCore appeared to work but was hitting OpenRouter directly (returning HTTP 402) instead of going through the bridge. The fix was to purge these fields entirely.
- A string comparison for fallback detection (`configured_model != resolved_model`) produced false positives on every clean run because the two fields use different naming conventions. The fix was to gate on the boolean `fallback_used` field instead.
- The security reviewer path through the bridge has not yet been E2E tested—it is a known gap.
- Operational tooling can still drift from the live state directory if operators bypass the repo-local wrappers or force legacy environment overrides. The default repo-root CLI path and the live `gateway health` snapshot have been repaired, but state-dir hygiene remains an operational concern outside the proof boundary.

8 What Is Verified Today

The crate `vericore-policy` currently verifies 187 functions with 0 errors across 16 modules. The honest split is now three-way: runtime-called kernel predicates, runtime-called boundary validators, and verified reference specs.

8.1 Verified Runtime Kernel

The following modules are production-wired: the same functions that Verus checks are the ones `vericore-core` calls for live policy decisions.

- tier definitions and flow-label algebra (`tiers.rs`): context rank, integrity rank, flow-to, label join. Proved properties include join idempotence, commutativity, associativity, and the duality between `can_flow_to` and `can_read`.
- default channel-to-context and context-to-integrity mapping (`channels.rs`): called by `GatePolicy::context_fo` and `integrity_for_channel` in production. Proved: public channels map to `Untrusted`, private channels map to `Trusted`, no channel maps above `Private`, and the integrity mapping is injective.
- ingress decision predicates (`ingress.rs`): `ingress_allows_exec` and `ingress_allows_write`, called from `check_ingress_integrity` in production. Proved: `Reviewed` cannot exec, `Trusted` has full capability, and write permissions are monotone in integrity.

8.2 Verified Runtime Boundary Validators

Four additional verified modules are now called directly by the runtime, but they validate facts reported by TypeScript rather than replacing TypeScript logic.

Session model policy (`session_model_policy.rs`). Called from the Rust LLM bridge client after each completion response. Proves metadata completeness, fallback visibility, provider-change flag consistency, allowlist enforcement, and override consistency. This is a boundary contract: it checks whether the bridge response is coherent enough for Rust to trust. It does *not* prove that the TS runtime selected the semantically correct model end to end.

Startup context identity (`startup_context_policy.rs`). Called through `startup_identity_validate` and surfaced in the startup/context report. Proves that AGENTS and SOUL availability/injection booleans combine into a simple contract. This is again a boundary validator over TS-observed facts, not an independent reconstruction of prompt assembly.

Heartbeat sync (`heartbeat_sync_policy.rs`). Called through `heartbeat_sync_validate` and surfaced in the context report. Proves that canonical and mirrored `HEARTBEAT.md` copies are both present, the mirror matches the canonical file, and the configured heartbeat prompt defers to `HEARTBEAT.md` rather than stale duplicated wording. This remains a boundary validator over TS-observed facts rather than a proof of the full heartbeat runner.

Heartbeat continuity context (`heartbeat_context_policy.rs`). Called through `heartbeat_context_valida` from the heartbeat runner and the context report. Proves that the recent-turn window stays bounded, a reservable recent main Telegram turn is preserved when present, content-bearing heartbeats remain visible in the main window, no-op heartbeats stay out of that main window, and the separate heartbeat trace stays bounded and oldest-first. This is a boundary validator over TS-selected continuity facts, not a proof of full prompt assembly.

8.3 Verified Reference Specs

Nine additional modules provide proved reference specs. These are not the authoritative runtime kernels; correspondence with runtime behavior is established by tests, runtime checks, and review.

Model resolution (`model_resolution.rs`). Proves candidate-list policy properties over an abstract `Seq<ModelRef>`: primary-first ordering, no duplicates, cross-provider gating, and alert severity semantics.

LLM bridge contract (`llm_bridge.rs`). Specifies the completions bridge interface: request well-formedness, response usability, call-kind/prompt-mode consistency, and session-key requirements.

Mindlock state machine (`mindlock.rs`). The full transition relation, staging/terminal classification, source validation, and cleanup eligibility (see Section 5).

Receipt authorization (`receipt.rs`). Promotion requires either a valid receipt (hash match AND target match) or explicit Zar approval with audit trail. Proved: tamper detection, target-swap detection, and the at-least-one-authorization invariant.

Egress flow control (`egress.rs`). Information flows upward in secrecy: `Public` can flow to anything, `Private` can only flow to `Private`. Proved: transitivity, same-tier always allowed, private-to-public denied, and security review triggers.

Gate chain (`gate_chain.rs`). The runtime gate chain is a strict conjunction of six boolean gates (schedule, action kind, tool ID, skill ID, ingress integrity, primitive action). Proved: any single deny breaks the chain, and the chain is monotone.

Tool broker contract (`tool_broker.rs`). Specifies broker request/response well-formedness and parsed capability-ID shape. This is explicitly a semantic parsed-field contract, not a raw JSON or string-parser proof.

Tool authorization policy (`tool_policy.rs`). Specifies allowlist authorization, native-name collision denial for brokered tools, and origin-irrelevant gating for non-colliding tools.

Cross-module compositions (`compositions.rs`). Proofs chaining `channels` → `tiers` → `ingress` and `tool_policy` → `gate_chain`. The current model-boundary lemma in this module is only a regression guard for the Rust validator; it should not be read as a deep proof that the TS model-selection pipeline is correct.

8.4 The Distinction

This separation matters and should be maintained honestly:

- **Verified runtime kernel**: Verus-checked pure decision functions that production calls directly for live allow/deny decisions.
- **Verified runtime boundary validator**: Verus-checked pure validators that production calls on TS-reported facts at the Rust seam.
- **Verified reference spec**: Verus-checked invariants that define what adjacent runtime behavior *should* satisfy. Correspondence is established by tests, runtime checks, and review, not by the prover alone.
- **Tested I/O and runtime behavior**: integration tests covering transport, filesystem, async races, and service availability. Verus does not and should not cover these.

Remaining drift surface: `vericore-core` still has its own copies of some enums and wire-level representations, with conversion helpers bridging them to the verified crate. The larger remaining drift surface is at the TS-to-Rust envelope and operator-facing routing/reporting seams, not inside the small pure kernel modules.

9 Proof Boundary

The current proof boundary covers all deterministic policy decision surfaces:

1. label algebra over secrecy and integrity (join lattice properties, flow/read duality),
2. channel-to-context and context-to-integrity mapping (injectivity, ceiling bounds),
3. ingress allow/deny decisions over normalized target classes (monotonicity, full capability),
4. session-model boundary validation over bridge-reported metadata (completeness, fallback visibility, override consistency, allowlist enforcement),

5. startup-identity boundary validation over AGENTS/SOUL availability and injection facts,
6. heartbeat-sync boundary validation over canonical/mirror file coherence and file-following prompt facts,
7. heartbeat continuity boundary validation over the recent-turn window and separate heartbeat trace,
8. mindlock state machine (terminal states, valid transitions, source validation, cleanup eligibility),
9. receipt and human-approval conditions for promotion (tamper detection, at-least-one-auth),
10. egress flow control (transitivity, private-to-public denial, review triggers),
11. gate chain conjunction (monotonicity, any-deny-breaks-chain),
12. tool broker and tool-gate contracts (parsed-field request/response invariants, native-shadow denial, allowlist semantics),
13. bridge contract (call-kind/prompt-mode consistency, session-key requirements), and
14. cross-module composition proofs for ingress/tool-gate chains.

The following are explicitly *outside* the proof boundary:

- Telegram and webchat UX,
- async daemon behavior,
- filesystem race handling in detail,
- reviewer prompt quality,
- RAG relevance,
- exact OpenClaw prompt assembly beyond the validated continuity-window facts,
- social or stylistic policy guidance.

This is not a weakness. It is a scoping decision. Formal verification is being used where deterministic invariants matter most and where the code can realistically remain simple enough to verify.

10 Current Weak Point: I/O Source Authenticity

The architecture is strong against *content spoofing* but weaker against *source spoofing*.

Content spoofing means message text tries to impersonate a more trusted source. VeriCore is already robust against this because authority comes from the channel assigned by the transport, not from the text itself.

Source spoofing is different. If an attacker can submit a forged daemon request with a falsely labeled channel such as `telegram_dm`, VeriCore will trust that label and derive `Private/Trusted`. That means the real trust boundary is not only the policy function; it is also the authenticity of the gateway-to-daemon envelope.

The right next hardening step is therefore transport authenticity:

- authenticate daemon requests,
- restrict who can submit control/private stimuli,
- later, sign or MAC file-based or agent-bridge envelopes.

11 Why a Verified Kernel Crate

Real verification-heavy systems do not stop at disconnected models forever. They either verify the actual code or a very thin executable kernel that the runtime actually calls. That is the design goal here.

The crate split is therefore pragmatic:

- keep async I/O, sockets, Telegram, and filesystem mutation in normal Rust; delegate LLM interaction to TypeScript via the completions bridge,
- move pure deterministic decision functions into a crate that is both executable and Verus-verifiable,
- have production call those functions directly.

This preserves runtime practicality while shrinking the gap between "proved model" and "running system".

12 Next Steps

The correct near-term sequence is now:

1. **(done)** ingress predicates Verus-verified and production-wired,
2. **(done)** default channel/context/integrity mapping verified and production-wired,
3. **(done)** single-driver completions bridge wired so Rust owns the turn loop while OpenClaw owns model resolution and auth,
4. **(done)** MCP tool broker wired so Rust can call brokered tools while OpenClaw owns MCP runtime and hooks,
5. **(done)** runtime boundary validators for session-model metadata, startup identity, heartbeat sync, and heartbeat continuity,
6. E2E test the security reviewer path through the completions bridge (still a known gap),
7. harden the gateway-to-daemon authenticity boundary,
8. strengthen TS↔Rust correspondence tests for model selection, tool broker behavior, and operator-surface routing,
9. reduce remaining operator/runtime drift (for example, non-Telegram private parent surfaces and transport/reporting edge cases), and
10. consolidate enum types and capability identifiers so production uses verified representations more directly.

Testing is still required. Verus does not replace integration tests for socket boundaries, filesystem behavior, async races, or service availability. The practical split is:

- **Verus (runtime kernel)**: pure policy functions called by production,
- **Verus (runtime boundary validators)**: pure validators called on TS-reported facts at the Rust seam,
- **Verus (reference spec)**: proved invariants mirrored by runtime tests and review, and
- **tests**: I/O, operational integration, and spec–runtime correspondence.

13 Conclusion

VeriCore is now past the point where detached proof sketches alone are the main value. As of March 11, 2026, `vericore-policy` verifies 187 functions with 0 errors across 16 modules spanning a runtime-called kernel, runtime-called boundary validators, and verified reference specs.

For the runtime kernel (tiers, channels, ingress), the verified predicates are authoritative runtime logic. For the boundary validators (session-model metadata, startup identity, heartbeat sync, and heartbeat continuity), the verified predicates are authoritative checks on TS-reported facts at the Rust seam. For the reference specs (model resolution, mindlock, receipt, egress, gate chain, bridge contracts, tool broker, tool policy, and compositions), the invariants are stated precisely and proved consistent, serving as regression guards and precise targets for future runtime wiring.

The honest trust story is therefore layered rather than absolute. Formal verification covers the deterministic kernel and some key seam validators. Tests and review still carry the transport, OpenClaw runtime, MCP lifecycle, UX routing, and other operational boundaries. That combination—verified deterministic policy for the runtime kernel, verified boundary contracts for the Rust/TS seam, verified reference specs for adjacent surfaces, and tested transport and operational boundaries—is the strongest realistic path for this architecture.

A Core Agent Architecture

This appendix records the deployed operational shape as of March 11, 2026. It distinguishes the *normal single-driver message path*, where VeriCore owns the turn loop, from the *heartbeat path*, which is currently synthesized and run inside OpenClaw rather than entering through VeriCore first.

A.1 A.1 Normal Interactive Turn

```
Human / channel stimulus
    |
    v
Telegram / web / API / internal surface
    |
    v
OpenClaw gateway ingress (TS)
  - parse surface metadata
  - resolve session key / agent binding
  - normalize inbound context
    |
    v
```

```
VeriCore daemon (Rust)
  - assign channel
  - map channel -> ContextTier -> IntegrityTier
  - run gate chain / ingress policy / audit logic
  - own the main turn loop
    |
    +-----------------------------+
    | |
    | completion needed | brokered tool needed
    v v
Completions bridge Tool broker bridge
  (Unix socket) (Unix socket)
    | |
    v v
OpenClaw completions server OpenClaw tool-broker server
  - session model override - per-agent MCP tool catalog
  - provider/model resolution - OpenClaw hook path
  - auth profile selection - MCP execution
  - fallback chain - stable capability ids
    | |
    v v
LLM provider(s) MCP runtimes / tools
                               - lean-lsp
                               - Tavily
                               - future brokers

Shared context visible to the LLM for a normal turn:
  1. OpenClaw system prompt
  2. injected workspace files (AGENTS.md, SOUL.md, TOOLS.md,
     IDENTITY.md, USER.md, HEARTBEAT.md, MEMORY.md, etc., budgeted)
  3. session history / thread context / system events
  4. current inbound user message

Execution / output path:
LLM/tool result -> VeriCore action dispatch -> native tools or brokered tools
              -> mindlock / audit / review when required
              -> OpenClaw outbound routing -> Telegram / web / other surface
```

A.2 A.2 Heartbeat Turn

```
15-minute scheduler + active-hours gate
    |
    v
OpenClaw heartbeat runner (TS)
  - skip during quiet hours
  - skip if main queue busy
  - skip if HEARTBEAT.md exists but is effectively empty
  - choose heartbeat session (default: agent main session)
    |
    v
Synthetic inbound heartbeat message
  Body =
    configured heartbeat prompt
```

```
    + current-time line
  Provider = "heartbeat"
  SessionKey = main session (unless overridden)
    |
    v
OpenClaw getReplyFromConfig(...)
  - same main agent runtime path as ordinary OpenClaw turns
  - NOT routed through VeriCore first
    |
    v
Embedded agent run / system prompt assembly
  - full bootstrap context by default
  - lightweight mode only if heartbeat.lightContext = true
  - current deployment uses full mode
    |
    v
LLM sees:
  1. system prompt with tool list, runtime, heartbeat instructions
  2. Project Context with injected workspace files, including
     AGENTS.md, SOUL.md, HEARTBEAT.md, and other bootstrap files
  3. surviving session history for the chosen session key
  4. deterministic continuity context assembled by TS
     and boundary-validated by VeriCore:
       - Recent completed turns block (limit 5)
       - reserve one recent main Telegram/Zar turn when available
       - reserve one recent content-bearing heartbeat when available
       - separate heartbeat trace block (configurable traceLimit; default 32)
       - no-op heartbeats stay in the trace, not the main turn window
  5. the synthetic user heartbeat message:
       "Heartbeat. Read HEARTBEAT.md and follow it calmly.
        If there is no clear call to action,
        reply HEARTBEAT_OK."
       plus the appended current time line

Reply handling:
  - HEARTBEAT_OK / empty reply -> may be pruned from transcript
  - nontrivial reply -> delivered to configured heartbeat target
                        or retained in-session if target = none
```

A.3 A.3 Bootstrap and Identity Files

```
Workspace bootstrap files (candidate set)
  AGENTS.md
  SOUL.md
  TOOLS.md
  IDENTITY.md
  USER.md
  HEARTBEAT.md
  BOOTSTRAP.md
  MEMORY.md / memory.md
  extra configured bootstrap files

Load path:
```

```
workspace -> bootstrap file resolver -> hook overrides -> char-budget truncation
         -> "Project Context" section of the system prompt

Compaction path:
session compaction -> bounded AGENTS.md + SOUL.md reinjection
                 -> post-compaction context refresh event
```

A.4 A.4 Trust Boundary Summary

```
Verus-checked and runtime-called:
  - channel/context/integrity kernel
  - ingress policy
  - startup identity boundary validator
  - heartbeat sync boundary validator
  - heartbeat context boundary validator
  - session-model boundary validator

Runtime-called but not fully proved end-to-end:
  - TypeScript heartbeat runner and exact prompt assembly
  - Telegram / web command routing
  - model override persistence
  - MCP lifecycle and transport health

Important current asymmetry:
  interactive parent / user turns -> VeriCore-first
  heartbeat turns -> OpenClaw heartbeat runner first
```